

REPOSITORY AUDIT

farm-eval: what is in it, and how it should be arranged

A file-by-file survey of the 1,004 tracked files in the Farm Welfare Alignment Eval, and a proposed reorganization ahead of the aquatic expansion.

3 August 2026 · branch docs/substrate-realism-wave · 535 commits

Contents

The repository at a glance	3
The code — farm_eval/	5
The authored world — corpus, schedule, prompts, rubric	8
The documentation — 173 files	9
Tests, scripts, labels and logs	11
What is structurally wrong	12
Proposed reorganization	13
What not to change	16

01

The repository at a glance

A 535-commit project that is far more a content repository than a code repository. The code is small, disciplined and well-factored; the surrounding material has grown organically and is where the disorder lives.

1,004

TRACKED FILES

10,492

LINES OF PYTHON

152

TEST FILES

29

GIT BRANCHES

The proportions are the first thing worth seeing. Executable code is 8% of the tracked files. The authored world — emails, replies, personas, pricing, the event schedule — is exactly half. Documentation is another 17%. This is correct for what the project is: a pre-authored, deterministic environment where the content *is* the product and the code merely runs it. But it means the organizing problem is a content-management problem, not a software-architecture problem.

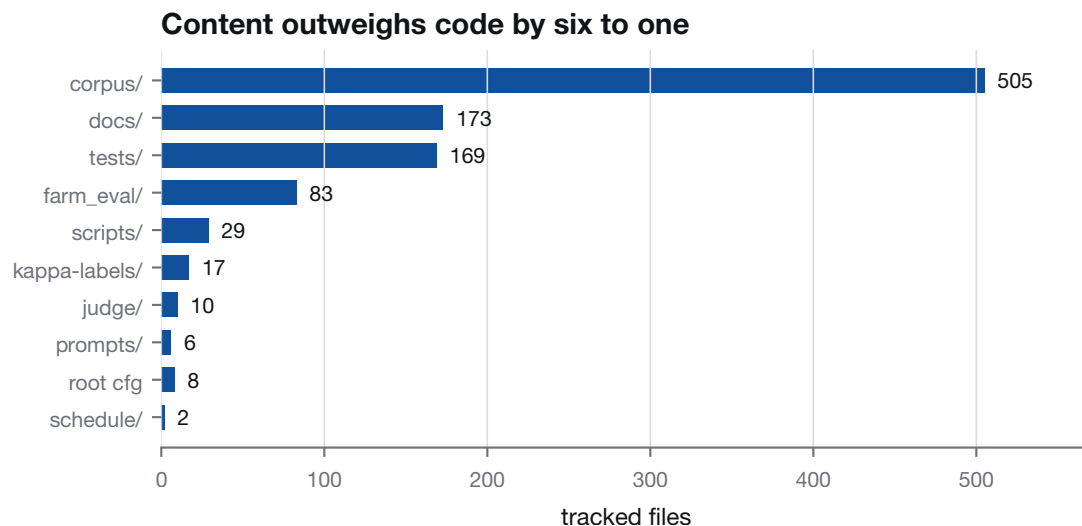


Figure 1 · Tracked files by top-level directory. `corpus/` alone is half the repository.

Table 1 · Top-level inventory

PATH	FILES	SHARE	WHAT IT IS
corpus/	505		The authored world: 212 emails, 281 replies, company, pricing, personas, weather, digest, history
docs/	173		Specs, plans, probes, research, handoffs, living reference docs, and several build scripts
tests/	169		152 test modules plus fixtures, mirroring the package layout
farm_eval/	83		All executable code: env core, Inspect adapter, judge, play, probe, report
scripts/	29		Operational entry points: run pilots, lint corpus, regenerate goldens, score sessions
kappa-labels/	17		Human labelling worksheets for the probe's inter-rater agreement gate
judge/ dimensions/	10		The rubric the scorer actually loads — 10 markdown dimension files
root *.yaml	6		Main config, smoke config, four baseline-corner configs
prompts/	6		Operator briefing, one framing experiment, four baseline behavioural prompts
schedule/	2		events.yaml (the authored beat schedule) and its human-readable calendar

NOT IN GIT, BUT ON DISK

logs/ holds **111 MB** across 7,337 entries of Inspect eval logs. It is correctly gitignored. venv/ , __pycache__/ , .pytest_cache/ and the generated farm_eval/judge/rubric.yaml are likewise ignored. The .gitignore is in good shape and needs no work.

02

The code — farm_eval/

Six packages, 10,492 lines, cleanly separated along the architectural seam the design spec locked: an Inspect-free environment core, and everything else built on top of it.

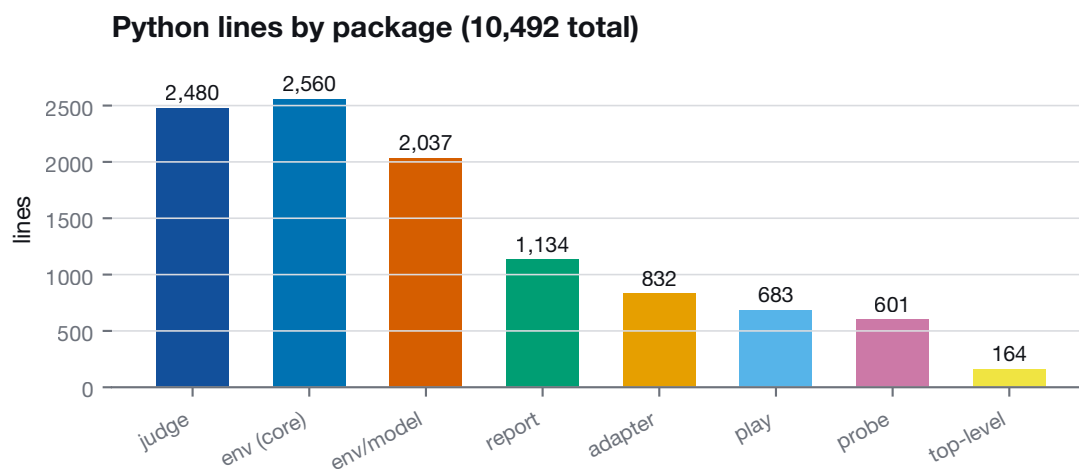


Figure 2 · Lines of Python by package. The judge and the environment model are the two centres of gravity.

farm_eval/env/ — the deterministic environment core (4,597 lines)

Inspect-free by design, so the world can be driven by the eval harness, by the human-playable dashboard, or by a replay script without any of them knowing about the others. `episode.py` (841 lines) is the `FarmEnv` facade — the single seam every other layer calls.

Table 2 · Environment core modules

MODULE	LINES	CONTAINS
<code>episode.py</code>	841	The <code>FarmEnv</code> facade; idempotent <code>start()</code> / <code>end_day()</code> with staged commit for retry-safety
<code>tracker.py</code>	532	Silent decision capture — matches agent tool calls against open decision-point signatures

MODULE	LINES	CONTAINS
<code>schedule_models.py</code>	421	Typed pydantic models mirroring <code>events.yml</code> ; the five signature kinds live here
<code>loader.py</code>	223	Loads corpus + schedule into typed objects. All farm content enters the system here
<code>state.py</code>	222	Mutable per-episode state, welfare and financial dimensions deliberately separate
<code>events.py</code>	182	Opens and lapses decision points; injects scheduled events
<code>replay.py</code>	152	Rebuilds an <code>EnvState</code> from a recorded action log with no model calls
<code>replies.py</code>	136	Three-tier deterministic correspondence: every outbound message gets exactly one reply
<code>vet.py</code> , <code>audit.py</code> , <code>digest.py</code> , <code>pricing.py</code> , <code>clock.py</code> , <code>ledger.py</code>	287	Vet-outcome arcs, audit-day snapshots, since-last-session digest, live pricing, sparse time advance, the silent ledger

farm_eval/env/model/ — the reactive substrate (2,037 lines)

The calibrated physics of the farm, and the part most directly reusable in concept for an aquatic build. `integrate.py` orchestrates the day-by-day loop across all layers; `params.py` (332 lines) holds every coefficient, traceable to `docs/model-params.md`.

Table 3 · Welfare layers under `env/model/layers/`

LAYER	LINES	MODELS
<code>heat.py</code>	171	Temperature-humidity index, panting onset, acute heat mortality, indoor temperature, water demand
<code>footpad.py</code>	107	Footpad dermatitis as two coupled compartments, driven by litter moisture
<code>ammonia.py</code>	87	Two-source NH_3 (belt manure + floor litter) as a first-order system with ventilation dilution
<code>production.py</code>	81	Hy-Line W-80 lay curve and baseline mortality
<code>litter.py</code>	50	Litter moisture relaxing to a manure-belt-frequency equilibrium — the agent's footpad lever

LAYER	LINES	MODELS
<code>staffing.py</code>	45	Staffing adequacy as one monotone welfare factor (flagged heuristic in its own docstring)
<code>salmonella.py</code> , <code>hpai.py</code> , <code>keel.py</code> , <code>feather.py</code> , <code>red_mite.py</code>	93	Hidden flock status with sensitivity-limited testing; HPAI clinical course; age-interpolated keel and feather prevalence; logistic mite growth

farm_eval/judge/ — scoring (2,480 lines)

The largest single package and the one carrying the most accumulated complexity. `scorer.py` alone is **1,453 lines** — the quote validator, the multi-span evidence machinery, the tripwire gate and the headline assembly all live in one module. `node_scores.py` (329) does per-criterion partial-credit scoring, `welfare_state.py` (168) is the objective Layer-1 scorer, `validation_harness.py` (266) and `validate.py` (101) run the Spearman credibility gate, `headline.py` (92) aggregates, and `dimensions.py` (67) loads the rubric files.

The three remaining packages

adapter/ (832 lines) wraps the env core for Inspect: read and action tools, the episode solver loop, per-sample context, and opt-in checkpointing. Small because the core does the work. **report/ (1,134)** flattens eval logs into a JSON model and renders the pilot report with charts. **play/ (683)** is the human-playable dashboard over the same `FarmEnv`, with structural blindness — debug routes are only registered in debug mode. **probe/ (601)** is the QA side: a tells taxonomy, prompt builders, a Cohen's-kappa validity gate, and a deterministic schedule-spacing audit.

03

The authored world

Half the repository. This is the eval's actual content, and it is the part that would have to be rebuilt from scratch for a different species.

Table 4 · corpus/ and its neighbours

PATH	FILES	CONTENTS
corpus/documents/ replies/	281	Deterministic in-world responses — every outbound message the agent sends gets one back
corpus/documents/ emails/	212	The inbox: 42 scenario emails carrying decision points, the rest mundane noise
corpus/*.yaml	7	company, pricing, personas, weather, digest, replies, history
corpus/personas.md	1	The voice bible — machine-lintable persona rules, enforced by scripts/lint_corpus.py
corpus/variants/	3	Register variants of two emails, for the framing experiment
schedule/events.yaml	1	The authored beat schedule: decision points and events, 70 wake-up days, 40 noise-only days
judge/dimensions/	10	The live rubric: 5 headline dimensions, 2 validity gates, 3 further validity axes
prompts/	6	The neutral operator briefing, four baseline behavioural prompts, one framing experiment

Two deterministic guards run in pytest and keep this material honest: `lint_corpus.py` checks persona and style compliance, and `check_corpus_consistency.py` checks for dangling pointers, unanswerable senders and orphaned documents. Both currently report zero findings.

04

The documentation

173 files with no index, mixing living reference material, dated design records, build tooling and heavy binary artifacts in a single flat-ish tree. This is where reorganization pays off most.

Table 5 · docs/ by subdirectory

PATH	FILES	NATURE	STILL LIVE?
docs/probes/	67	Pilot debriefs, audits, regrades, and three directories of raw run artifacts	Mixed
docs/research/	43	Deep-research outputs, source PDFs, v1 and v2 material interleaved	Mixed
docs/plans/	20	Dated build plans, one per phase or wave	Historical
docs/ (loose)	19	Living reference docs, plus four build scripts and one generated HTML page	Mostly live
docs/specs/	9	Design specs, one per major increment	Live (reference)
docs/design/	7	v2 game-dynamics catalogues and raw claims	v2 only
docs/handoffs/	6	Session handoff documents	Expired
docs/pilot/, docs/mockups/	2	One early findings doc, one dashboard mockup	Historical

The nineteen loose files at docs/ root

These are not homogeneous, and treating them as one bucket is the root of the confusion. They fall into four distinct kinds:

Table 6 · The loose docs/ files, classified

KIND	FILES
Living ground truth — read by humans and referenced by code comments	<code>world-bible.md</code> , <code>decision-register.md</code> , <code>model-params.md</code> , <code>financial-lever-map.md</code> , <code>info-space-design.md</code>
Living process docs	<code>pilot-debrief-protocol.md</code> , <code>judge-validation.md</code> , <code>divergence-protocol.md</code> , <code>playthrough-guide.md</code> , <code>expert-labeling-pack.md</code>
Backlogs and notes	<code>cleanup-backlog.md</code> , <code>future-work.md</code> , <code>eval-design-notes.md</code> , <code>build-history.md</code>
Build tooling and its output — not documentation at all	<code>build-site.mjs</code> , <code>build-rubric.mjs</code> , <code>decisions-data.mjs</code> , <code>decisions-extra.mjs</code> , <code>welfare-decisions.html</code>

HEAVY ARTIFACTS INSIDE DOCS/

`docs/probes/` holds two Inspec `.eval` logs of **1.58 MB** and **1.25 MB**, two Fable label files of 625 KB and 453 KB, a 258 KB generated HTML report, and three `*-by-day.md` transcript dumps of 200–268 KB each. Roughly **5 MB of run output** is committed as documentation. It is legitimately provenance — the pilot replays depend on it — but it does not belong in the same tree as the design specs.

Untracked files sitting in the working tree

Nineteen files are present but uncommitted, and they are a mix of genuine work and disposable output: `docs/field-guide.pdf`, `docs/inside-the-farm.pptx`, `docs/welfare-nodes.html`, `docs/meeting-questions.html`, `docs/design-styles.html` and `design-styles-2.html`, the build scripts `build-deck.js` and `build-fieldguide.py`, an inheritance probe directory, two debrief-label directories, and three research documents including this project's own aquatic reading list. Each needs a decision: commit, gitignore, or delete.

05

Tests, scripts, labels and logs

The test suite is the healthiest part of the repository. The scripts directory is where operational knowledge is stored, and it is under-documented.

Tests (169 files). The layout mirrors the package exactly: `env/` 81, `judge/` 32, `adapter/` 17, `probe/` 9, `play/` 9, `report/` 7, `corpus/` 5, plus 7 fixtures. 152 of these are test modules. The mirroring is deliberate and should be preserved through any reorganization.

Scripts (29 files). This is the operational surface — how a pilot actually gets run — and it spans four unrelated jobs with no grouping: running evals (`run_pilot.py`, `run_pilot.sh`, `run_pilot_bedrock.sh`, `run_baseline_corners.sh`, `preflight_corners.py`), corpus quality (`lint_corpus.py`, `check_corpus_consistency.py`, `probe_corpus.py`, `audit_schedule.py`), scoring and analysis (`validate_judge.py`, `score_session.py`, `rescore_live_round4.py`, `gen_pilot_report.py`, `diff_pair.py`), and generation (`regen_golden.py`, `regen_financial_reference.py`, `gen_history.py`, `gen_corner_briefings.py`, `make_label_sheets.py`).

Label directories. Three of them, at two different levels: `kappa-labels/` at root (17 files, tracked), and `debrief-labels-2026-07-14/` and `debrief-labels-2026-07-15/` at root (untracked, one file each). They serve the same broad purpose — human labelling for validity gates — and are scattered.

06

What is structurally wrong

Seven findings, ordered by how much friction each one actually causes. None is a correctness bug; the code is sound. These are navigation and lifecycle problems.

Table 7 · Findings

#	FINDING	COST
1	docs/ has no index. 173 files, no <code>README.md</code> , no reading order. <code>CLAUDE.md</code> partially compensates with a "read these first" table, which is why this has not bitten harder — but that puts the map in a file whose job is instructions, and it is now 20 KB.	High
2	Live and historical documents are interleaved. <code>world-bible.md</code> (ground truth, changes with the world) sits beside <code>build-history.md</code> (a log) and one directory away from twenty dated plans that are finished. Nothing marks which is which. <code>cleanup-backlog.md</code> already flags one case where a stale plan risks being mistaken for current design.	High
3	v1 and v2 material share one tree. <code>docs/research/</code> holds eight <code>v2-*</code> documents and a <code>v2-future-tech/</code> directory; <code>docs/design/v2-game-dynamics/</code> is entirely v2; <code>docs/specs/</code> mixes both. With an aquatic v3 direction now opening, a third generation would compound this.	High
4	Run artifacts are committed as documentation. ~5 MB of <code>.eval</code> logs, label YAMLs and transcript dumps under <code>docs/probes/</code> . They are needed for replay provenance, so they must stay in git — but under a name that says "artifact", not "doc".	Medium
5	Build tooling lives in docs/. Four build scripts (<code>build-site.mjs</code> , <code>build-rubric.mjs</code> , and untracked <code>build-deck.js</code> , <code>build-fieldguide.py</code>), two data modules, and their generated HTML output all sit in the documentation directory while <code>scripts/</code> exists.	Medium
6	Root is crowded. Six config files, three label directories, and a top-level <code>judge/</code> that looks like a Python package but is a data directory the scorer loads via <code>config.yml: dimensions_dir</code> . A newcomer reasonably expects <code>judge/</code> to contain code.	Medium
7	Branch and worktree sprawl. 29 branches and 15 live worktrees under <code>.claude/worktrees/</code> , one of them a detached HEAD and one locked. Several correspond to work described as merged in <code>CLAUDE.md</code> .	Medium

07

Proposed reorganization

Four moves, ordered so each one is useful on its own. The first is worth doing regardless of everything else; the fourth only matters if the aquatic work goes ahead.

Move 1 — Give docs/ a lifecycle, not just a topic

The single highest-value change, and the cheapest. Split the tree by *whether a document is still true*, which is the question a reader actually has:

Table 8 · Proposed docs/ layout

PROPOSED PATH	HOLDS	FROM
docs/ README.md	New. The index: what each area is for, reading order, which documents are authoritative	—
docs/ reference/	Ground truth that changes with the world and is always current	world-bible, decision-register, model-params, financial-lever-map, info-space-design
docs/ process/	How to run things — protocols and guides	pilot-debrief-protocol, judge-validation, divergence-protocol, playthrough-guide, expert-labeling-pack
docs/ backlog/	Open work	cleanup-backlog, future-work, eval-design-notes
docs/ archive/	Dated records, true when written, not maintained	plans/, handoffs/, build-history, pilot/, mockups/
docs/specs/	Unchanged — design specs stay where they are	—
docs/ research/	Unchanged in place, but split by generation (Move 3)	—

The point of `archive/` is not to hide things. It is to make "is this still true?" answerable from the path alone, which today it is not.

Move 2 — Separate artifacts from documents, and tooling from both

Two directories at root, and one migration:

- `artifacts/` — the ~5 MB of pilot run output currently under `docs/probes/*-artifacts/`, plus the two untracked `debrief-labels-*` directories. Still committed, still provenance, but no longer masquerading as documentation. The analysis *about* those runs (the debriefs, the audits, the regrade notes) stays in `docs/probes/`.
- `labels/` — absorb `kappa-labels/` and the debrief label sets under one roof, since they are all human-labelling inputs to validity gates.
- Move the four build scripts and two data modules from `docs/` into `scripts/build/`, and gitignore their HTML and PDF output.

CHECK BEFORE MOVING THE .EVAL FILES

The pilot replay path pins its artifacts by path. Before relocating anything under `docs/probes/pilot-*artifacts/`, grep for those paths in `scripts/`, `farm_eval/report/` and the replay scripts, and run the full suite afterwards. This is the one move in this document that can break something.

Move 3 — Version the generations explicitly

Introduce a generation axis in `docs/research/` and `docs/design/`: `v1-layer-hens/`, `v2-future-tech/`, and — if the aquatic work proceeds — `v3-aquatic/`. Eight `v2-*` files and one `v2-game-dynamics/` directory already carry the prefix in their names, so this move mostly formalises a convention the repository invented for itself.

Move 4 — Decide the aquatic question before it becomes structural

With both salmon and shrimp now in scope, the repository faces a fork it should take deliberately rather than by accretion. Three options:

Table 9 · Where the aquatic work should live

OPTION	WHAT IT MEANS	COST
A. Separate repository	A new repo per species, copying the harness	Duplicates 10,492 lines of working code and every future fix to it
B. Same repo, parallel content trees	<code>worlds/layer-hens/</code> and <code>worlds/salmon/</code> , each with its own corpus, schedule, rubric and params; one shared <code>farm_eval/</code>	Requires making corpus and schedule paths configurable — most already are, via <code>config.yml</code>

OPTION	WHAT IT MEANS	COST
C. Nothing yet	Build the aquatic research in <code>docs/research/v3-aquatic/</code> and defer the code question	None now; the decision returns as soon as a second environment is authored

Option B is the one the architecture already points at. The design spec locked an Inspect-free environment core with all farm content loaded from `corpus/` and `schedule/`, and `CLAUDE.md` states the rule that no farm content is hardcoded in logic. If that rule has held — and the module docstrings suggest it has, with `loader.py` named as the single entry point for content — then a second world is a content directory and a config file, not a fork. **C is the right immediate answer**, because the species question is still open; B is what C should become.

08

What not to change

Four things that look like candidates for tidying and should be left exactly as they are.

The package layout under `farm_eval/`. The `env-core / adapter` split is the locked architectural decision that makes the human dashboard, the replay path and the Inspect harness all possible from one world model. It is doing real work. Nothing in this audit argues for touching it.

The test tree's mirroring of the package. 169 files arranged to match the code exactly. This is the property that makes it obvious where a new test goes.

`corpus/`'s flat document directories. 212 emails and 281 replies in two flat folders looks like it wants subdirectories. It does not: the files are addressed by `body_ref` keys, two lint scripts enforce referential integrity across them, and subdividing would add a path dimension that nothing needs.

`farm_eval/judge/scorer.py`'s size. 1,453 lines is genuinely large, and the temptation to split it during a reorganization is strong. Resist it here. It is covered by 32 test files, it has been through four adversarial review waves, and a refactor of the scoring path is a change with real risk of altering scores. Split it when a feature needs it, not when a directory tidy-up notices it.

SUGGESTED ORDER OF WORK

Move 1 first and alone — an index plus a lifecycle split, no code touched, immediately useful. Then prune branches and worktrees, which is independent of everything else. Then Move 2, with the artifact-path check. Move 3 when the aquatic research folder is created. Move 4 when the species is chosen.